

Design and Implementation of a Nonlinear Backstepping Controller for an Inverted Pendulum

Team 29

Youssef Mohamed Abodeb
dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt
55-1031

Somaya Magdy
dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt
55-5027

Bassam Walid
dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt
55-0499

Yassin Hesham
dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt
55-1555

Styven Hany
dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt
55-10067

Mostafa Shakweer
dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt
56-29282

Abstract—The Furuta pendulum is a classical benchmark problem in control engineering, characterized by nonlinear dynamics, strong coupling, and underactuation. This paper presents the modeling, simulation, and stabilization of a Furuta pendulum using a Lyapunov-based backstepping control strategy. The nonlinear equations of motion are derived using the Euler–Lagrange formulation and implemented in MATLAB/Simulink. A backstepping controller is designed to stabilize the pendulum around the unstable upright equilibrium by systematically regulating the pendulum angle, pendulum angular velocity, and arm motion based on Lyapunov stability theory. A Stateflow-based supervisory logic is incorporated to manage controller activation and indicate balanced operation. Simulation results demonstrate rapid convergence to the upright equilibrium and stable behavior without sustained oscillations, validating the effectiveness of the proposed backstepping control approach for underactuated nonlinear systems.

Keywords— Furuta pendulum, Nonlinear control, Backstepping, Lyapunov stability, Stateflow

I. INTRODUCTION

The Furuta pendulum, also known as the rotary inverted pendulum, is a classical benchmark problem in control engineering and mechatronics education. It represents a highly nonlinear, underactuated mechanical system in which an unstable equilibrium must be stabilized using indirect actuation. The system consists of a horizontal rotating arm driven by a motor and a pendulum rod attached to the arm, rotating in a vertical plane. Since only the arm is actuated while the pendulum remains unactuated, control must rely on the dynamic coupling between the two degrees of freedom [1].

Stabilizing the Furuta pendulum at the upright equilibrium is a challenging control problem due to the system's nonlinear dynamics, strong coupling, and inherent instability. Linear control techniques are typically insufficient to guarantee stability over a meaningful operating region, which motivates the use of nonlinear control methods with formal stability guarantees.

In this project, a Lyapunov-based backstepping nonlinear control strategy is developed to stabilize the Furuta pendulum around the upright equilibrium. The system dynamics are derived using the Euler–Lagrange formulation and implemented in MATLAB/Simulink. The backstepping controller is designed to ensure local asymptotic stability by systematically regulating the pendulum angle, pendulum

angular velocity, and arm motion based on Lyapunov stability theory.

Reference [1] is used to describe the Furuta pendulum system and its modeling, while this work focuses exclusively on Lyapunov-based backstepping stabilization

II. SYSTEM MODELING AND DYNAMICS

The subject of this section is to describe the mathematical model of the Furuta pendulum and its main parameters, which are necessary for realizing the Lyapunov-based backstepping stabilization control.

A. Mechanical Description and Derivation

The Furuta pendulum, also referred to as the inverted rotary pendulum, is a three-dimensional mechanical system, in contrast to the planar cart–pendulum configuration. The system possesses two degrees of freedom, with an actuating torque applied to the horizontal arm, while the pendulum remains unactuated. The pendulum is attached to the arm such that their axes of rotation are perpendicular to each other, resulting in a strongly coupled and underactuated system.

The mathematical model of the system is derived using the Euler–Lagrange equations of motion. This formulation provides a systematic approach for modeling multibody mechanical systems with nonlinear dynamics. Viscous damping due to joint friction is included in the model to more accurately represent the physical behavior of the system. The resulting equations of motion form the basis for the subsequent control design and simulation.

B. Non-Linear Dynamic Equations

$$M(x)\ddot{q} + C(x, \dot{x}) + G(x) + F(\dot{x}) = \tau$$

Where:

$$q = \begin{bmatrix} \theta_r \\ \theta_k \end{bmatrix}, \tau = \begin{bmatrix} \tau \\ 0 \end{bmatrix}$$

1) Mass(Inertia)Matrix

$$M(\theta_k) = \begin{bmatrix} I_r + m_k l_r^2 + m_k l_k^2 \sin^2(\theta_k) & m_k l_r l_k \cos(\theta_k) \\ m_k l_r l_k \cos(\theta_k) & I_k + m_k l_k^2 \end{bmatrix}$$

2) Coriolis/Centrifugal Vector

$$C(x, \dot{x}) = \begin{bmatrix} m_k l_k^2 \sin(\theta_k) \cos(\theta_k) \dot{\theta}_k^2 \\ 0 \end{bmatrix}$$

3) Gravity Vector

$$G(x) = \begin{bmatrix} 0 \\ m_k g l_k \sin(\theta_k) \end{bmatrix}$$

4) Viscous Damping

$$F(\dot{x}) = \begin{bmatrix} B_r \dot{\theta}_r \\ B_k \dot{\theta}_k \end{bmatrix}$$

5) Arm Equation

$$u = M_{11} \ddot{\theta}_r + M_{12} \ddot{\theta}_k + C_1 + F_1$$

$$u = (I_r + m_k l_r^2 + m_k l_k^2 \sin^2(\theta_k)) \ddot{\theta}_r + m_k l_r l_k \cos(\theta_k) \ddot{\theta}_k + m_k l_k^2 \sin(\theta_k) \cos(\theta_k) \dot{\theta}_k^2 + B_r \dot{\theta}_r$$

6) Pendulum Acceleration Equation

$$(I_k + m_k l_k^2) \ddot{\theta}_k + m_k l_r l_k \cos(\theta_k) \ddot{\theta}_r + m_k g l_k \sin(\theta_k) + B_k \dot{\theta}_k = 0$$

$$\ddot{\theta}_k = \frac{m_k g l_k \sin(\theta_k) + B_k \dot{\theta}_k}{I_k + m_k l_k^2} - \frac{m_k l_r l_k \cos(\theta_k)}{I_k + m_k l_k^2} \ddot{\theta}_r$$

$$\ddot{\theta}_k = a + b \ddot{\theta}_r$$

7) Arm Acceleration Equation

$$\ddot{\theta}_r = \frac{u - m_k l_k^2 \sin \theta_k \cos \theta_k \dot{\theta}_k^2 - B_r \dot{\theta}_r + \frac{m_k l_r l_k \cos \theta_k (m_k g l_k \sin(\theta_k) + B_k \dot{\theta}_k)}{I_k + m_k l_k^2}}{(I_r + m_k l_r^2 + m_k l_k^2 \sin^2(\theta_k) - \frac{(m_k l_r l_k \cos \theta_k)^2}{I_k + m_k l_r^2})}$$

C. State Vector

The system is represented using four state variables:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} \theta_r \\ \dot{\theta}_r \\ \theta_k \\ \dot{\theta}_k \end{bmatrix}$$

where:

State	Symbol	Meaning
x_1	θ_r	Rotary arm angular position(rad)
x_2	$\dot{\theta}_r$	Rotary arm angular velocity(rad/s)
x_3	θ_k	Pendulum angular position measured from upright vertical(rad)
x_4	$\dot{\theta}_k$	Pendulum angular velocity(rad/s)

D. Kinematic Equations

State Space Representation:

$$\begin{aligned} \dot{x}_1 &= x_2 \\ \dot{x}_2 &= \ddot{\theta}_r \\ \dot{x}_3 &= x_4 \\ \dot{x}_4 &= \ddot{\theta}_k \end{aligned}$$

With:

$$\begin{bmatrix} \ddot{\theta}_r \\ \ddot{\theta}_k \end{bmatrix} = M^{-1}(x_3) \begin{bmatrix} u \\ 0 \end{bmatrix} - C(x) - G(x) - F(x)$$

Final State-Space Form:

$$\dot{x}_1 = x_2$$

$$\dot{x}_2 = \begin{bmatrix} M^{-1}(x_3) \begin{bmatrix} u \\ 0 \end{bmatrix} - C(x) - G(x) - F(x) \end{bmatrix}$$

$$\dot{x}_3 = x_4$$

$$\dot{x}_4 = \begin{bmatrix} M^{-1}(x_3) \begin{bmatrix} u \\ 0 \end{bmatrix} - C(x) - G(x) - F(x) \end{bmatrix}$$

E. System Parameters

Parameter	Symbol	Value
Arm length	l_r	0.1 m
Arm mass moment of inertia	I_r	0.00005 kg m ²
Arm friction coefficient	B_r	0.0002 kg m ² s ⁻¹
Pendulum mass	m_k	0.03 kg
Distance from pendulum center of mass to rotation axis	l_k	0.1 m
Pendulum mass moment of inertia	I_k	0.0004 kg m ²
Pendulum friction coefficient	B_k	0.00008 kg m ² s ⁻¹
Pendulum position	θ_k	rad
Arm position	θ_r	rad
Torque	τ	Nm

III. CONTROL STRATEGY

Lyapunov-Based Backstepping Controller

Desired equilibrium:

θ_r does not need to converge to a specific value

$\theta_k = \pi$ or $-\pi$

$\dot{\theta}_k = 0$

$\dot{\theta}_r = 0$

1) Subsystem 1 : Pendulum Position State:

$$z_1 = \theta_k - \pi$$

This is the angle error

Goal: $z_1 \rightarrow 0$

Dynamics: $\dot{z}_1 = \dot{\theta}_k$

We cannot control $\dot{\theta}_k$ directly so we introduce a desired behavior for it

First Lyapunov Function

We choose : $V_1 = \frac{1}{2} z_1^2$

This represents the energy of the angle error.

$$\dot{V}_1 = z_1 \dot{\theta}_k$$

This is not stable yet so we introduce a Virtual Control:

Define a desired pendulum velocity :

$$\dot{\theta}_k^* = -k_1 z_1 \quad (k_1 > 0)$$

If this were achieved :

$$\dot{V}_1 = -k_1 z_1^2 < 0$$

So the pendulum angle would stabilize

2) Subsystem 2: Pendulum Velocity

Since we cannot directly enforce $\dot{\theta}_k = \dot{\theta}_k^*$

We define a new error:

$$z_2 = \dot{\theta}_k - \dot{\theta}_k^* = \dot{\theta}_k + k_1 z_1$$

Now the system becomes:

$$\dot{z}_1 = -k_1 z_1 + z_2$$

If $z_2 \rightarrow 0$, then $z_1 \rightarrow 0$

Extended Lyapunov Function

We now include both errors:

$$V_2 = \frac{1}{2} z_1^2 + \frac{1}{2} z_2^2$$

Derivative:

$$\dot{V}_2 = z_1(-k_1 z_1 + z_2) + z_2 \dot{z}_2$$

$$\dot{V}_2 = -k_1 z_1^2 + z_1 z_2 + z_2 \dot{z}_2$$

We still need to control $\dot{z}_2$

$$\ddot{\theta}_k = a + b \ddot{\theta}_r$$

So

$$\dot{z}_2 = \ddot{\theta}_k + k_1 \dot{z}_1 = a + b \ddot{\theta}_r + k_1(-k_1 z_1 + z_2)$$

By substituting $\dot{z}_2$ we get

$$\dot{V}_2 = -k_1 z_1^2 + z_1 z_2 + z_2(a + b \ddot{\theta}_r + k_1(-k_1 z_1 + z_2))$$

$$\dot{V}_2 = -k_1 z_1^2 + z_2(a + b \ddot{\theta}_r + k_1 z_2 + z_1(1 - k_1^2))$$

From here we can say that

$$z_2(a + b \ddot{\theta}_r + k_1 z_2 + z_1(1 - k_1^2)) = -k_2 z_2$$

And by rearranging we get the desired arm acceleration:

$$\ddot{\theta}_r^* = \frac{-k_1(-k_1 z_1 + z_2) - k_2 z_2 - a}{b}$$

$$\dot{V}_2 = -k_1 z_1^2 - k_2 z_2^2 < 0$$

So pendulum position and velocity are stabilized

3) Subsystem 3: Arm Velocity

Desired arm velocity:

$$\dot{\theta}_r^* = 0$$

Error:

$$z_3 = \dot{\theta}_r - 0$$

Final Lyapunov Function

$$V_3 = V_2 + \frac{1}{2} z_3^2$$

Derivative:

$$\dot{V}_3 = -k_1 z_1^2 - k_2 z_2^2 + z_3(\ddot{\theta}_r - \ddot{\theta}_r^*)$$

Final Control Law:

We chose the real arm acceleration as:

$$\ddot{\theta}_r = \ddot{\theta}_r^* - k_3 z_3 \quad (k_3 > 0)$$

Then:

$$\dot{V}_3 = -k_1 z_1^2 - k_2 z_2^2 - k_3 z_3^2 < 0$$

This proves asymptotic stability

4) Calculation on input u

From here we can calculate the torque input using the values of $\ddot{\theta}_r$ and $\ddot{\theta}_k$ obtained above

$$u = M_{11}\ddot{\theta}_r + M_{12}\ddot{\theta}_k + C_1 + F_1$$

IV. SYSTEM IMPLEMENTATION (SIMULINK/STATEFLOW)

The overall Furuta pendulum control architecture constitutes was then simulated on MATLAB SimuLink using two main MATLAB blocks; one for the dynamics of the system and one for the control.

1) SimuLink

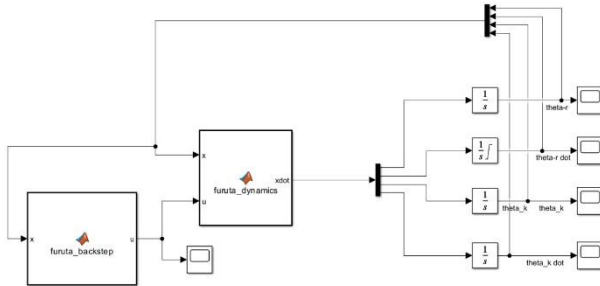


Figure 1 SimuLink Model

The Dynamics block had an outputted the four states which went into integrators and gave back their integrals which went back again as input along with the input u and both are used to calculate the next states.

The other MATLAB function contained the control loop that uses the integrated states as inputs to calculate the input u . The controller has three gains that were adjusted to make the system give the fastest and smoothest response while still respecting the real life model constraints.

The first constraint is within the control loop where the torque u final value was restricted so it can have a maximum of ± 1 Nm. Which was the maximum torque that could be produced by the motor.

The second constraint can be seen in figure 2 where the angle of the pendulum has a saturation limit of ± 8.9 rad/s which is approximately equal to 110 RPM which is maximum RPM of the motor used.

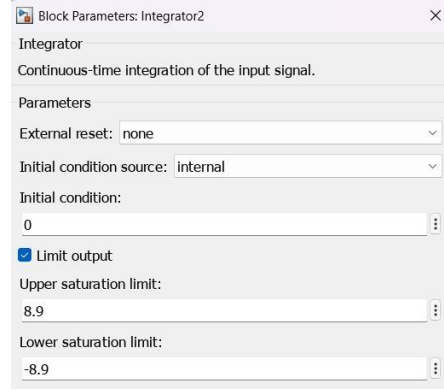


Figure 2 Saturation Limit

2) Stateflow Logic

Figure shows the closed-loop Simulink implementation of the Furuta pendulum system using a Lyapunov-based backstepping controller. The nonlinear plant dynamics are implemented using a MATLAB Function block that computes the state derivatives based on the applied control torque. The controller continuously computes the control input using full state feedback and applies it to the plant. The resulting angular accelerations are integrated to obtain the arm and pendulum positions and velocities, which are fed back to the controller, forming a fully closed-loop nonlinear control system.

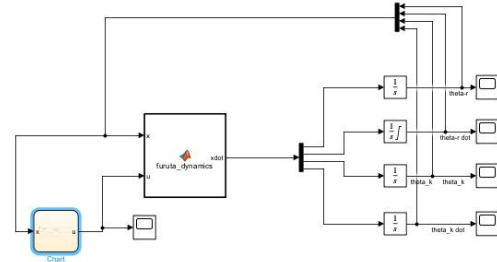


Figure 3 SimuLink-Stateflow implementation

Figure 4 illustrates the Stateflow-based supervisory logic used to manage the operating modes of the backstepping stabilization process. The chart monitors the pendulum state and governs the transition between initialization, active stabilization, and balanced operation. This discrete logic ensures that the stabilizing controller is applied only when appropriate, improving robustness while maintaining continuous nonlinear control

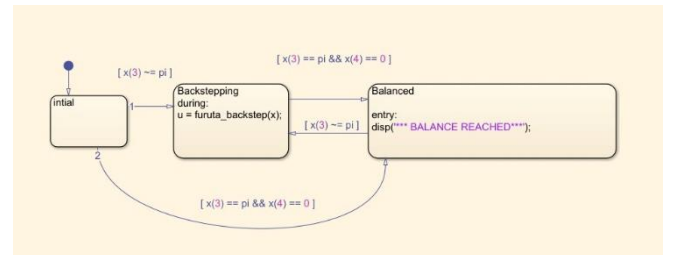


Figure 4 Stateflow Chart

The Stateflow chart consists of three discrete states: *Initial*, *Backstepping*, and *Balanced*, each representing a distinct operating mode of the stabilization process.

- **Initial State:**
This state represents the system startup condition. No active control action is applied, and the state serves as an initialization phase before stabilization is enabled.
- **Backstepping State:**
In this state, the Lyapunov-based backstepping controller is activated. The control input is continuously computed based on the system state and applied to the plant to stabilize the pendulum around the upright equilibrium.
- **Balanced State:**
This state indicates successful stabilization of the pendulum at the upright position. The system remains in this state as long as the pendulum angle and angular velocity remain within predefined tolerance bounds. Deviations from this condition trigger a return to the backstepping state to re-establish stability.

V. RESULTS AND DISCUSSION

This section presents the results obtained from the MATLAB/Simulink simulation of the Furuta Pendulum, validating the implemented non-linear control algorithms.

1) SimuLink Results

The initial downwards position of the system is at $\theta_k = 0$, here the system is hanging downwards and it's the hardest challenge of the controller to bring it to the upwards position at $\pm \pi$. Figures 5,6,7,8 show the system states with all initial values at zero to make the system hard to control.

$$x = \begin{bmatrix} \theta_r \\ \dot{\theta}_r \\ \theta_k \\ \dot{\theta}_k \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

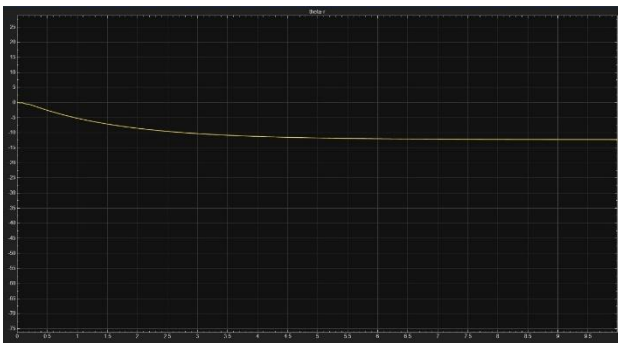


Figure 5: θ_r

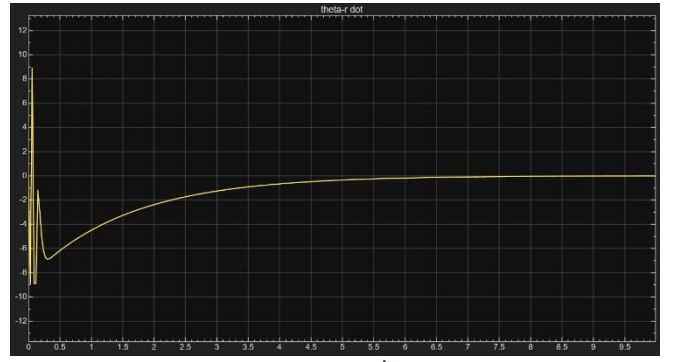


Figure 6: $\dot{\theta}_r$

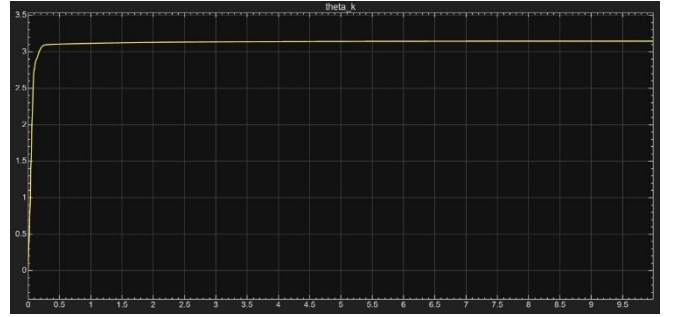


Figure 7: θ_k

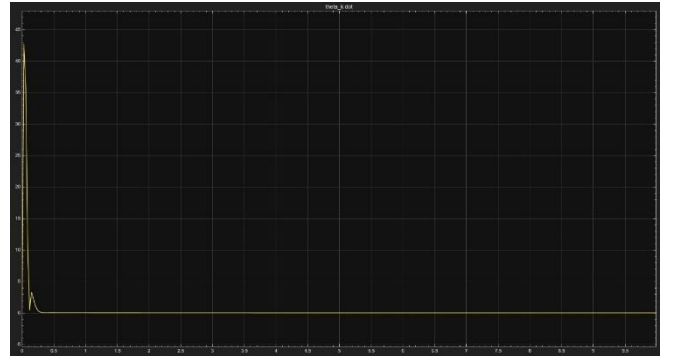


Figure 8: $\dot{\theta}_k$

Figure 5 shows the pendulum position reaching stability at a random value as it is not necessary to control so it reaches specific position. Its control is done indirectly by making the pendulum velocity reach 0 which can be seen in figure 4.

Figure 7 shows the pendulum reaching equilibrium at Ω in approximately 0.5 seconds which shows how powerful backstepping control can be.

In figure 8 the velocity of the pendulum can be seen reaching equilibrium following at the same time as the pendulum stabilized which proves the system dynamics are correct.

The System was then tested using different initial conditions and to show that the controller works in all ranges. The graphs below show the pendulum reaching equilibrium with different initial conditions.

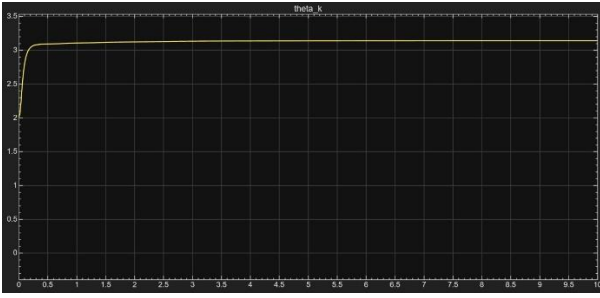


Figure 9: $\theta_k = 2$

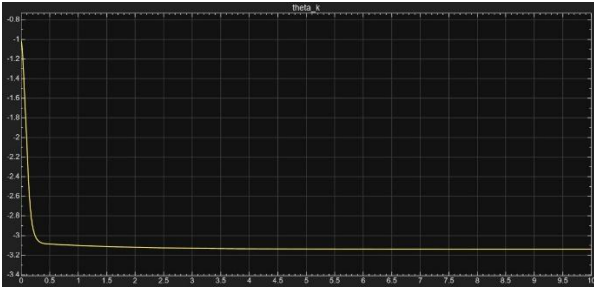


Figure 10: $\theta_k = -1$

The system is also shown in open loop when u is equal to zero and the initial value of the pendulum at different values. And it can be seen oscillating around the zero which is the downwards position. The graph in figure 9 can be seen decreasing in value as the system is damped

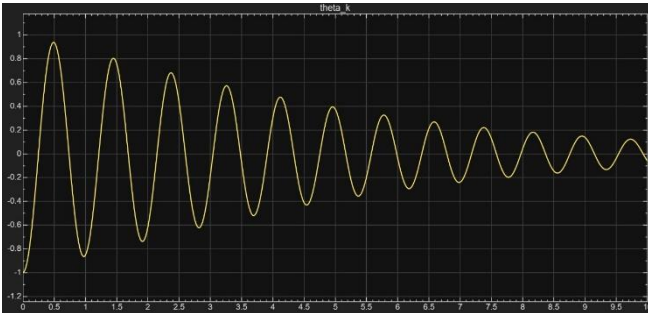


Figure 11: open loop oscillating $\theta_k = -1$

In figure 12 the system starts at π which is the upwards position, and it can be seen stable for a few seconds before falling down and oscillating around the zero

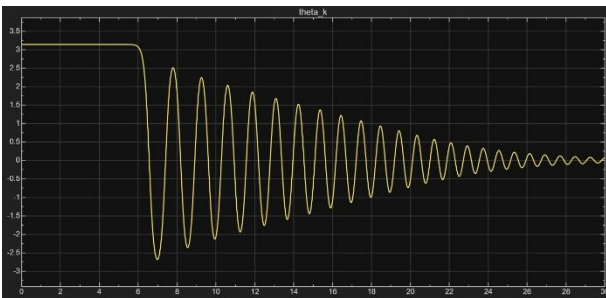


Figure 12: open loop starting at $\theta_k = \Omega$

2) Stateflow Results

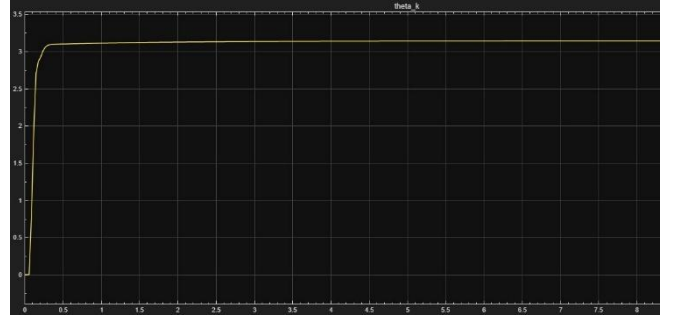


Figure 13 Pendulum angular position θ_k response under Stateflow backstepping control.

Figure 13 shows the pendulum angular position θ_k under the backstepping stabilization controller. The pendulum rapidly converges to the upright equilibrium at $\theta_k = \pi$ and remains stable without oscillations. This response confirms correct operation of the backstepping controller and the associated Stateflow supervisory logic.

VI. EXPERIMENTAL SETUP

A. Hardware Components

1. jgb37-520 dc motor
 2. md10c motor driver
 3. Current Sensor Module ACS712-5A
 4. Arduino Uno
 5. Raspberry pi 5
 6. Amron rotary encoder E6B2-CW6C with 2000 P/R
- The raspberry pi 5 and Arduino are connected via a usb serial connection.**

B. Setup

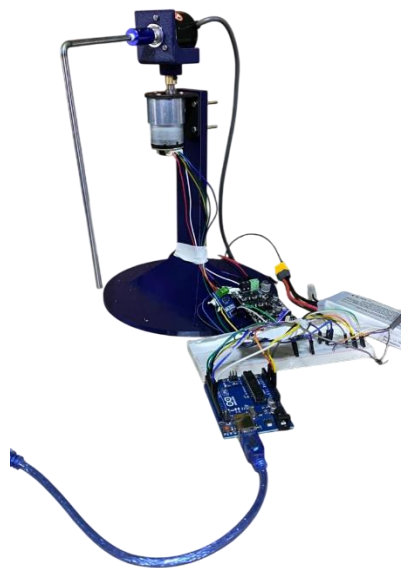


Figure 14 System in initial downward position

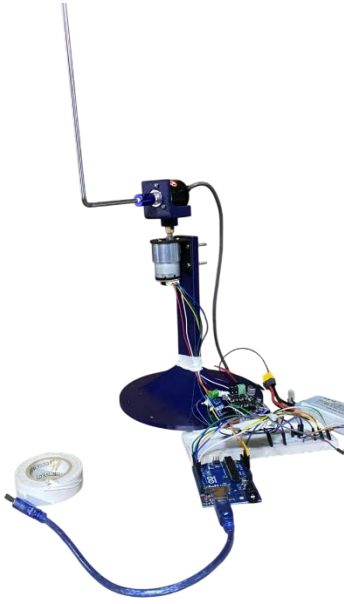


Figure 15 System in upright position

VII. REFERENCES

- [1] P. Seman, B. Rohal'-Ilkiv, M. Juhás and M. Salaj, "Swinging up the Furuta pendulum and its stabilization via model predictive control," *Journal of Electrical Engineering*, vol. 64, no. 3, pp. 152–158, 2013, doi: 10.2478/jee-2013-0022.